ValleyScript: Its Like Static Typing

Cormac Flanagan

September 27, 2007

Abstract

We formalize the ES4 notions of `like` types and `wrap` operators for a lambda-calculus with ES4-style objects, to better understand these concepts and to clarify what guarantees can be provided by the verifier in strict mode. We also formalize `Self`.

1 Language Overview

We consider the implementation of a *gradual typed* language that supports both typed and untyped terms, which interoperate in a flexible manner. We begin by defining the syntax of terms and types in the language: see Figure 1. In addition to the usual terms of the lambda calculus (variables, abstractions, and application), the language also includes constants and expressions to create, dereference, and update objects. It also includes `is` expressions, which check that a value has a particular type, and `wrap` expressions, which, if necessary, wrap the given value to ensure that it behaves like it has that type.

The type language is fairly rich. In addition to base types (`Int` and `Bool`), function types, and object types, the language includes additional types related to gradual typing. The type $*$ is (roughly) a top type, and indicates that no static type information is known. The type `like` T describes values whose value components match T, but whose type components may be more vague than T, due to the presence of the type $*$. (Due to imperative constructs, that matching-value guarantee does not persist, and so `like` types are helpful for debugging but do not provide strong guarantees.)

The type `Self` may be used only within an object type T, and refers to the *allocated type* of the current object. The type rules for `Self` are somewhat subtle, since it may denote any subtype of the object type T.

Figure 1: Syntax

$e ::=$		*Terms:*
	c	constant
	x	variable
	$\lambda x{:}S.\ e : T$	abstraction
	$e\ e$	application
	$\{\bar{l} = \bar{e}\} : T$	object expression
	$e.l$	member selection
	$e.l()$	member invocation
	$e.l := e$	member update
	e `is` T	dynamic type check
	e `wrap` T	wrapping a value
$c ::=$		*Constants:*
	n	integer constant
	b	boolean constant
$S, T ::=$		*Types:*
	`Int`	integers
	`Bool`	booleans
	$S \to T$	function type
	$\{\bar{l} : \bar{T}\}$	object types
	$*$	dynamic type
	`like` T	like types
	`Self`	self type

2 Evaluation

We next describe the evaluation semantics of the language. The set of values in the language is given by:

$v ::=$		*Values:*
	c	constant
	$\lambda x{:}S.\ e : T$	abstraction
	v `wrapped` T	wrapped value
	a	object address
$o ::=$		*Object value:*
	$\{\bar{l} = \bar{v}\} : T$	object value

A *object store* σ maps object addresses a to object values of the form $\{\bar{l} = \bar{v}\} : T$. Every value has an *allocated type* according to the function $ty_\sigma(v)$:

$$\begin{array}{rcll} ty_\sigma(n) & = & \texttt{Int} & \\ ty_\sigma(b) & = & \texttt{Bool} & \\ ty_\sigma(\lambda x{:}S.\ e : T) & = & (S \to T) & \\ ty_\sigma(v \texttt{ wrapped } T) & = & T & \\ ty_\sigma(a) & = & T & \text{if } \sigma(a) = \{\ldots\} : T \end{array}$$

An evaluation context is:

$$\begin{array}{rcl} C & ::= & \bullet \mid \bullet\ t \mid v\ \bullet \mid \bullet \texttt{ wrap } T \\ & & \mid \{\bar{l} = \bar{v}, l = \bullet, \bar{l} = \bar{e}\} : T \mid \bullet.l \mid \bullet.l() \mid \bullet.l := e \mid v.l := \bullet \end{array}$$

A *state* is a pair of an object store and a current expression. The evaluation relation on states is defined by the rules in Figure 2. The rule [E-Alloc] requires an explicit object type with a type for each field, but those field types could simply be $*$. (Note, fields can never be deleted in our semantics.) Thus, the necessary object type could always be locally inferred from the object expression.

In general, states never contain types that have free occurrences of `Self`. Instead, whenever we pull a component type S out of an object with allocated type $T = \{l : S, \ldots\}$, we replace all occurrences of `Self` in S with the type T.

Several rules refer to the judgement v $\texttt{is}_\sigma$ T, which checks if the value v matches the type T: see Figure 3. The type `Self` should not occur free in T. This judgement relies on two subtype-like relations on types.

The judgement $S <: T$ (*S is a subtype of T*) checks if every value of type S can be assigned to a variable of type T. The type `Self` can occur free in both S and T. The type $*$ is a top type.

The judgement $S \sim: T$ (*S is compatible with T*) extends the subtyping relation with a more flexible interpretation of dynamic types; in particular, the type $*$ is compatible with any type. The compatibility judgement is not transitively closed. In particular, we have that `Int` $\sim: *$ and $* \sim:$ `Bool`, but the judgement `Int` $\sim:$ `Bool` does not hold. The rule [S-Like] for `like` types switches from checking subtyping to checking compatibility.

Lemma 1 *The subtyping judgement is reflexively-transitively closed.*

Lemma 2 (Preservation under subtyping) *If* v $\texttt{is}_\sigma$ S *and* $S <: T$ *then* v $\texttt{is}_\sigma$ T.

TBP.

Two types are *indistinguishable* if they are indistinguishable under the subtyping, compatibility, and `is` relations.

Conjecture: The types `like` T and `like like` T are indistinguishable.

Figure 2: Evaluation Rules

$$\sigma, C[(\lambda x{:}S.\ t : T)\ v] \longrightarrow \sigma, C[t[x := v]\ \texttt{is}\ T] \quad \text{if } v\ \texttt{is}_\sigma\ S \qquad \text{[E-Beta1]}$$

$$\sigma, C[(w\ \texttt{wrapped}\ (S \to T))\ v] \longrightarrow \sigma, C[(w\ (v\ \texttt{wrap}\ S))\ \texttt{wrap}\ T] \qquad \text{[E-Beta2]}$$

$$\sigma, C[v\ \texttt{is}\ T] \longrightarrow \sigma, C[v] \quad \text{if } v\ \texttt{is}_\sigma\ T \qquad \text{[E-As]}$$

$$\sigma, C[v\ \texttt{wrap}\ T] \longrightarrow \sigma, C[w] \qquad \text{[E-Wrap]}$$
$$\text{where } w = \begin{cases} v & \text{if } ty_\sigma(v) <: T \\ v\ \texttt{wrapped}\ T & \text{if } v\ \texttt{is}_\sigma\ T \end{cases}$$

$$\sigma, C[\{l_i = v_i^{i \in 1..n}\} : T] \longrightarrow \sigma[a := (\{l_i = v_i\} : T)], C[a] \qquad \text{[E-Alloc]}$$
$$\text{where } T = \{l_i : T_i^{i \in 1..n}\}$$
$$\text{and } v_i\ \texttt{is}_\sigma\ T_i[\texttt{Self} := T]$$
$$\text{and } a \text{ fresh}$$

$$\sigma, C[a.l] \longrightarrow \sigma, C[v] \quad \text{if } \sigma(a) = \{l = v, \ldots\} : T \qquad \text{[E-Get1]}$$

$$\sigma, C[(w\ \texttt{wrapped}\ T).l] \longrightarrow \sigma, C[(w.l)\ \texttt{wrap}\ S[\texttt{Self} := T]] \qquad \text{[E-Get2]}$$
$$\text{where } T = \{l : S, \ldots\}$$

$$\sigma, C[a.l()] \longrightarrow \sigma, C[(a.l)\ a] \qquad \text{[E-Call]}$$

$$\sigma, C[a.l := v] \longrightarrow \sigma[a, l := v], C[v] \qquad \text{[E-Assign1]}$$
$$\text{where } \sigma(a) = \{\ldots\} : T$$
$$\text{and if } T = \{l : S, \ldots\}$$
$$\text{then } v\ \texttt{is}_\sigma\ S[\texttt{Self} := T]$$

$$\sigma, C[(w\ \texttt{wrapped}\ T).l := v] \longrightarrow \sigma, C[w.l := (v\ \texttt{wrap}\ S[\texttt{Self} := T])] \qquad \text{[E-Assign2]}$$
$$\text{where } T = \{l : S, \ldots\}$$

Figure 3: Relations on Types

Dynamic type check $\boxed{v \ \texttt{is}_\sigma \ T}$

$$\frac{ty_\sigma(v) <: T}{v \ \texttt{is}_\sigma \ T} \quad \text{[Is-Ok]}$$

$$\frac{\sigma(a) = \{l_i = v_i^{i \in 1..n+m}\} : S \qquad v_i \ \texttt{is}_\sigma \ \texttt{like} \ T_i \ \text{ for } i \in 1..n}{a \ \texttt{is}_\sigma \ \texttt{like} \ \{l_i : T_i^{i \in 1..n}\}} \quad \text{[Is-Obj]}$$

$$\frac{ty_\sigma(v) \sim: T \qquad v \text{ not an object address}}{v \ \texttt{is}_\sigma \ \texttt{like} \ T} \quad \text{[Is-NonObj]}$$

Subtyping $\boxed{S <: T}$

$$\frac{}{T <: T} \quad \text{[S-Refl]}$$

$$\frac{T_1 <: S_1 \qquad S_2 <: T_2}{(S_1 \to S_2) <: (T_1 \to T_2)} \quad \text{[S-Arrow]}$$

$$\frac{T_i <: S_i \qquad S_i <: T_i \qquad \text{for } i \in 1..n}{\{l_i : S_i^{i \in 1..n+m}\} <: \{l_i : T_i^{i \in 1..n}\}} \quad \text{[S-Obj]}$$

$$\frac{}{T <: *} \quad \text{[S-Dyn]}$$

$$\frac{S \sim: T}{S <: \texttt{like} \ T} \quad \text{[S-Like]}$$

$$\frac{S <: T}{\texttt{like} \ S <: \texttt{like} \ T} \quad \text{[S-Like-Covariant]}$$

Compatible Types $\boxed{S \sim: T}$

Includes all of the above rules, and also:

$$\frac{}{* \sim: T} \quad \text{[C-Dyn]}$$

3 Strict Mode Type System

In a traditional statically typed language, the type system fulfills two goals:

1. It detects certain errors at compile time.
2. It guarantees what kinds of values are produced by certain expressions, which enables run-time check elimination.

The situation in ES4 is somewhat different, because of two reasons. First, in `standard` mode, we would like to eliminate run-time checks where possible, using a type-based analysis, without reporting any compile-time type errors. Second, `like` types weaken the guarantees provided by `strict` mode. For these reasons, we actually present *two* type systems.

For example, if a variable x has type `like` $\{f : \texttt{Int}\}$, then $x.f$ could actually return a value of any type. Nevertheless, $x.f$ would be expected to produce values of type `Int`, and so the expression $x.f.g$ would yield a compile-time type error. Thus, we say that $x.f$ has type `Int`, but this type is *only* a statement of intent; it does not guarantee what kinds of values are returned by that expression, and so cannot be used for run-time check elimination.

The strict mode type system is based on a judgement $E \vdash e : T$, stating that expression e has type T in environment E. Note that the type T *only* indicates that e is intended to produce values of type T; there are no guarantees here, due to the use of `like` types. However, this intent can be still used to detect type errors at compile time.

Figure 4: Type Rules for Strict Mode

Type rules $\boxed{E \vdash t \,:\, T}$

$$\frac{(x : \texttt{like}^*\ T) \in E}{E \vdash x \,:\, T} \quad \text{[T-Var]}$$

$$\frac{}{E \vdash c \,:\, ty(c)} \quad \text{[T-Const]}$$

$$\frac{E, x : S \vdash e \,:\, T' \qquad T' <: T}{E \vdash (\lambda x{:}S.\ e : T) \,:\, (S \to T)} \quad \text{[T-Fun]}$$

$$\frac{E \vdash t_1 \,:\, (S \to T) \qquad E \vdash t_2 \,:\, S' \qquad S' <: S}{E \vdash (t_1\ t_2) \,:\, T} \quad \text{[T-App1]}$$

$$\frac{E \vdash t_1 \,:\, * \qquad E \vdash t_2 \,:\, S'}{E \vdash (t_1\ t_2) \,:\, *} \quad \text{[T-App2]}$$

$$\frac{E \vdash t \,:\, S}{E \vdash t\ \texttt{is}\ T \,:\, T} \quad \text{[T-As]}$$

$$\frac{E \vdash t \,:\, S}{E \vdash t\ \texttt{wrap}\ T \,:\, T} \quad \text{[T-Wrap]}$$

$$\frac{E \vdash t_i \,:\, S_i \qquad S_i <: T_i[\texttt{Self} := T] \qquad T = \{l_i : T_i^{i \in 1..n}\}}{E \vdash (\{l_i = t_i^{i \in 1..n}\} : T) \,:\, T} \quad \text{[T-Alloc]}$$

$$\frac{E \vdash e \,:\, T \qquad T = \{l : S, \ldots\}}{E \vdash e.l \,:\, S[\texttt{Self}^+ := T, \texttt{Self}^- := ..T..]} \quad \text{[T-Get1]}$$

$$\frac{E \vdash e \,:\, *}{E \vdash e.l \,:\, *} \quad \text{[T-Get2]}$$

$$\frac{E \vdash e \,:\, T \qquad T = \{l : \texttt{Self} \to S, \ldots\}}{E \vdash e.l() \,:\, S[\texttt{Self}^+ := T, \texttt{Self}^- := ..T..]} \quad \text{[T-Call1]}$$

$$\frac{E \vdash e \,:\, *}{E \vdash e.l() \,:\, *} \quad \text{[T-Call2]}$$

$$\frac{\begin{array}{c}E \vdash e_1 \,:\, T \qquad T = \{l : S, \ldots\} \qquad E \vdash e_2 \,:\, S' \\ S' <: S[\texttt{Self}^- := T, \texttt{Self}^+ := ..T..]\end{array}}{E \vdash e_1.l := e_2 \,:\, S} \quad \text{[T-Set1]}$$

$$\frac{E \vdash e_1 \,:\, * \qquad E \vdash e_2 \,:\, S}{E \vdash e_1.l := e_2 \,:\, S} \quad \text{[T-Set2]}$$

4 Check Optimization

We now sketch a type-based analysis that statically identifies dynamic type checks that can be eliminated. We introduce the following additional "`safe`" expression forms, for which run-time checks are unnecessary.

$e ::=$		*Expressions:*
	$\ldots$	expressions mentioned earlier
	`safe` x	safe variable
	`safe` $\lambda x{:}S.\ e : T$	safe abstraction
	`safe` $e\ e$	safe application
	`safe` $\{\bar{l} = \bar{e}\} : T$	safe object allocation
	`safe` $e.l$	safe member selection
	`safe` $e.l()$	safe member invocation
	`safe` $e.l := e$	safe member update
$v ::=$		*Values:*
	$\ldots$	values mentioned earlier
	`safe` $\lambda x{:}S.\ e : T$	safe abstraction

It is straightforward to formulate the operational semantics of the extended language.

Figure 5 presents *optimization rules*, which verify that `safe` ... occurs in correct places; it is straightforward to reformulate the analysis to infer these `safe` annotations.

Some rules, such as [O-Get-Safe], replace contravariant occurrences of `Self` with a minimal type $\bot$, whihc is guaranteed to always be a subtype of the current object type. Thus, extracting a field (whose type has no contravariant occurrences of `Self`) is safe; extracting a method (a function whose argument type is `Self`) may also be safe, but the static argument type of the resulting function is $\bot$, so invoking that function is always unsafe, and its argument must be checked dynamically. Via [O-Set-Safe], updating a member is in general safe only if its type does not contain covariant occurrences of `Self`.

The following lemmas remain to be proven.

Lemma 3 (No Failure) *For any term e, there is some placement of* `safe` *annotations into e yielding e' such that $\emptyset \Vdash e' : T$ for some T.*

Lemma 4 (Soundness) *If $\emptyset \Vdash e : T$, then* `safe` *operation in e can never get stuck.*

Figure 5: Type Rules for Optimization

Type rules $\boxed{E \Vdash t : T}$

$$\frac{(x : T) \in E}{E \Vdash \texttt{safe}\ x : T} \quad \text{[O-Var-Safe]}$$

$$\frac{}{E \Vdash x : *} \quad \text{[O-Var-Unsafe]}$$

$$\frac{}{E \Vdash c : ty(c)} \quad \text{[O-Const]}$$

$$\frac{E, x : S \Vdash e : T' \qquad T' <: T}{E \Vdash \texttt{safe}\ (\lambda x{:}S.\ e : T) : (S \to T)} \quad \text{[O-Fun-Safe]}$$

$$\frac{E, x : S \Vdash e : T'}{E \Vdash (\lambda x{:}S.\ e : T) : (S \to T)} \quad \text{[O-Fun-Unsafe]}$$

$$\frac{E \Vdash t_1 : (S \to T) \qquad E \Vdash t_2 : S' \qquad S' <: S}{E \Vdash \texttt{safe}\ (t_1\ t_2) : T} \quad \text{[O-App-Safe]}$$

$$\frac{E \Vdash t_1 : S \qquad E \Vdash t_2 : S'}{E \Vdash (t_1\ t_2) : *} \quad \text{[O-App-Unsafe]}$$

$$\frac{E \Vdash t : S}{E \Vdash t\ \texttt{is}\ T : T} \quad \text{[O-As]}$$

$$\frac{E \Vdash t : S}{E \Vdash t\ \texttt{wrap}\ T : T} \quad \text{[O-Wrap]}$$

$$\frac{E \Vdash t_i : S_i \qquad S_i <: T_i[\texttt{Self} := T] \qquad T = \{l_i : T_i^{i \in 1..n}\}}{E \Vdash \texttt{safe}\ (\{l_i = t_i^{i \in 1..n}\} : T) : T} \quad \text{[O-Alloc-Safe]}$$

$$\frac{E \Vdash t_i : S_i}{E \Vdash (\{l_i = t_i^{i \in 1..n}\} : T) : T} \quad \text{[O-Alloc-Unsafe]}$$

$$\frac{E \Vdash e : T \qquad T = \{l : S, \ldots\}}{E \Vdash \texttt{safe}\ e.l : S[\texttt{Self}^+ := T, \texttt{Self}^- := \bot]} \quad \text{[O-Get-Safe]}$$

$$\frac{E \Vdash e : S}{E \Vdash e.l : *} \quad \text{[O-Get-Unsafe]}$$

$$\frac{E \Vdash e : T \qquad T = \{l : \texttt{Self} \to S, \ldots\}}{E \Vdash \texttt{safe}\ e.l() : S[\texttt{Self}^+ := T, \texttt{Self}^- := \bot]} \quad \text{[O-Call-Safe]}$$

$$\frac{E \Vdash e : S}{E \Vdash e.l() : *} \quad \text{[O-Call-Unsafe]}$$

$$\frac{E \Vdash e_1 : T \qquad T = \{l : S, \ldots\} \qquad E \Vdash e_2 : S' \qquad S' <: S[\texttt{Self}^- := T, \texttt{Self}^+ := \bot]}{E \Vdash \texttt{safe}\ (e_1.l := e_2) : S} \quad \text{[O-Set-Safe]}$$

$$\frac{E \Vdash e_1 : * \qquad E \Vdash e_2 : S}{E \Vdash e_1.l := e_2 : S} \quad \text{[O-Set2]}$$

5 Sugar

$$\begin{aligned}
\lambda x{:}(\texttt{wrap}\ S).\ e : T &= \lambda x{:}(\texttt{like}\ S).\ \texttt{let}\ x : S = (x\ \texttt{wrap}\ S)\ \texttt{in}\ e : T \\
\lambda x{:}S.\ e : (\texttt{wrap}\ T) &= \lambda x{:}S.\ (e\ \texttt{wrap}\ T) : T \\
\texttt{wrap}\ (\lambda x{:}S.\ e : T) &= \lambda x{:}(\texttt{wrap}\ S).\ e : T
\end{aligned}$$